

Retrieval Augmented Generation System for German Historical Speeches

Ram Binay Gupta
Marburg University

FB12 Mathematics and Computer Science Department
Guptar@students.uni-marburg.de
3848283

Mohammed Afaan Sajjad Hussain Shaikh
Marburg University

FB12 Mathematics and Computer Science Department
Shaikhm@students.uni-marburg.de
3844933

Abstract

This project presents a secure, containerized Retrieval Augmented Generation (RAG) system for question answering on historical speeches. The system combines modern large language models with an adaptive retrieval pipeline to ensure accurate, context grounded answers. Using LangGraph, we created an agentic RAG workflow that dynamically routes queries, rewrites questions via self reflection, and decomposes complex queries into manageable sub questions. At each stage, an LLM based grader evaluates document relevance and tests for hallucinations, enabling iterative refinement. The knowledge corpus (English and German speech transcripts) is segmented with a robust recursive text splitter. We employ Arctic-Embed 2.0 [18] for multilingual dense embeddings, achieving high retrieval quality in both English and German without sacrificing performance. Documents are indexed in Elasticsearch and retrieved via cosine similarity scoring. When initial vector retrieval provides insufficient results (detected by the LLM grader), a hybrid search fallback combines keyword (BM25) and semantic scores to improve result relevance elastic [15]. The system supports two LLMs: Llama 3.2 (a fast-response model for standard queries) and Qwen2.5-7B (a more powerful model used for agentic reasoning in complex cases). A Next.js frontend allows users to select the LLM, and all components (FastAPI backend, frontend, Ollama model server, and Elasticsearch) run in Docker containers for portability and security isolation. We detail the RAG pipeline architecture, discuss design choices (e.g. recursive vs. semantic chunking), and evaluate the system on a set of automatically LLM generated historical speech evaluation set. Experimental results show the adaptive RAG approach improves answer quality (BLEU, ROUGE-L, BERTScore) and reduces hallucinations.

1 Introduction

Question answering over historical speeches is a challenging task that benefits from Retrieval Augmented Generation. Many important speeches are lengthy, multilingual, and contain nuanced historical context that exceeds the memorized knowledge of modern LLMs. RAG provides a solution by grounding LLM responses in actual speech transcripts, ensuring factual accuracy and allowing the model to cite what was actually said. This is crucial in the historical domain, where precision and trustworthiness of quotes or contexts are important. By retrieving relevant passages

from speeches and feeding them into an LLM, the system can answer questions such as “What did the speaker emphasize about economic policy on 1935 August speech?” with direct evidence, rather than relying on potentially hallucinated answer. However, vanilla RAG[10] pipelines have limitations when dealing with complex, multi part queries often posed about historical content. Traditional RAG follows a linear retrieve then generate flow, which can stumble if the retrieved documents are irrelevant, if the query is poorly formulated, or if multiple reasoning steps are [13]. For example, a question like “Compare the speeches of speaker in 1934 and 1941 on Economy” requires gathering information from multiple years and comparing them wherein a single step retrieval and generation may fail to produce relevant result. Also, LLM may hallucinate the result. To address these issues, we attempt to employ agentic RAG[10] that uses decision point and feedback loops into the QA process.

In this paper, we describe the architecture of our RAG system for historical speeches, emphasizing how LangGraph is used to implement an agentic workflow with conditional logic. We outline the motivation for each component: from the choice of text splitting strategy to the selection of embeddings and models. The pipeline performs LLM driven self reflection – like, rewriting the user query if initial retrievals fail and can decompose complex questions into simpler sub queries which are answered in turn and then composed into a final answer. This follows the paradigm of agentic RAG, where autonomous agents (powered by LLMs) handle planning, retrieval, and synthesis in multiple iterations.

We also highlight how the system ensures security and reproducibility through containerization. All components run within Docker, enabling deployment on controlled infrastructure without data leakage. This is important for handling sensitive historical corpus in a secure environment. The containerized setup simplifies scaling and maintenance, encapsulating dependencies for the front end, backend, LLM servers, and vector database.

The remainder of this paper is structured as follows. Section 2 describes the system architecture in detail, including the adaptive RAG pipeline and its components. Section 3 discusses the document processing techniques, comparing the recursive chunking with semantic chunking approach. Section 4 covers the multilingual embedding and retrieval strategy using Arctic-Embed 2.0 and Elasticsearch, along with the hybrid search fallback mechanism. In Section 5, we present the two LLMs integrated (Llama 3.2 and Qwen2.5-7B) and explain their roles in normal vs. agentic modes. Section 6 outlines the evaluation methodology, metrics, and results, demonstrating the system’s performance on QA tasks. Section 7 describes the deployment architecture, including how the Dockerized services interact. Finally, Section 8 concludes with key findings, limitations, and future work.

1.1 Motivation

The study of historical speeches plays a crucial role in understanding past events, societal changes, and cultural developments. As digital archives of such texts continue to expand, manual analysis becomes increasingly impractical and time consuming. There is a clear need for automated solutions capable of efficiently extracting relevant information and providing informed answers to complex questions. By integrating document retrieval techniques with advanced language models, it is possible to create systems that support large scale, accurate, and context aware analysis of historical speeches. This approach seeks to enhance research capabilities and facilitate deeper insights into historical texts.

1.2 Project Goals

- Apply Retrieval Augmented Generation (RAG) to answering user questions over historical speeches.
- Improve document retrieval to maximize answer relevance and reduce hallucinations.
- Integrate retrieved context effectively with large language models.
- Analyze the influence of retrieval quality on generated answers.
- Evaluate RAG performance using metrics.
- Investigate limitations and potential of RAG in historical text analysis.

2 Related Work

Adaptive retrieval augmented generation systems have recently incorporated mechanisms for self reflection and query decomposition to tackle complex queries. One notable approach is Self Reflective RAG, which trains the LLM to assess its own outputs using special “reflection” tokens and only retrieve additional evidence when necessary [3]. This self critique mechanism helps reduce hallucinations by adaptively deciding when to consult external documents. Other adaptive strategies treat retrieval as an iterative reasoning process: for example, an Adaptive RAG model classifies question complexity to choose between single step or multi step retrieval, aiming to be more effective than static retrieval while more efficient than exhaustive multi hop reasoning [5]. In parallel, query decomposition methods address multi hop questions by breaking the original query into simpler sub queries that can be answered with separate evidence. This technique assembles disjoint facts from multiple sources and then merges them for the final answer, substantially improving recall and answer accuracy.

Unlike prior work that relies on trained classifiers or specialized routing modules to distinguish between simple, multi hop, or summarization type queries, our approach uses the LLM itself as the routing agent. Instead of training a task-specific classifier to detect query complexity, ambiguity, or summarization intent, we leverage the LLM’s inherent semantic reasoning to perform query routing through natural-language planning prompts. This design avoids additional training overhead, adapts more flexibly to domain shifts, and simplifies deployment in multilingual or evolving corpora. Moreover, using the LLM for routing allows our pipeline to exploit context-aware decision making something static classifiers struggle to approximate resulting in more reliable handling of both straightforward queries and deeply contextual historical analysis tasks.

Hybrid retrieval strategies have also been explored to combine the strengths of dense and sparse search. Dense vector retrievers excel at finding semantically related content but may miss exact keyword matches, whereas sparse retrievers (e.g. BM25) provide high precision for keyword overlap. Recent RAG systems therefore integrate both, for instance, experiments show that BM25 tends to optimize precision while embedding based search offers broader recall, suggesting that a hybrid of the two yields more complete and relevant results [17]. One effective implementation is to weight lexical BM25 and semantic embedding scores equally to balance keyword and semantic matching [14]. Such hybrid retrieval schemes have been found to retrieve more pertinent context and reduce answer hallucinations compared to relying on either method alone.

Another crucial aspect is how documents are split into chunks for retrieval. Naive fixed length splitting can disrupt context, so recent work examines recursive, semantic, and hybrid chunking strategies. Recursive chunking leverages document structure (e.g. splitting by sections, then paragraphs) to maintain local coherence, while semantic chunking groups sentences into chunks based on embedding similarity to ensure topical unity [9]. Each method alone has drawbacks in balancing coherence and chunk size.

Parallel to chunk optimization, researchers have introduced LLM-driven query rewriting to refine the queries themselves. Instead of using the raw user question, the system first has an LLM rephrase or expand the query with relevant keywords and domain specific terminology, so that it better aligns with the indexed knowledge. Ma et al. [2023] demonstrate such a “rewrite-then-retrieve” paradigm: a trainable query rewriter transforms the user’s query to more closely match the corpus language, which significantly improves the accuracy of retrieved passages and thus the factual correctness of the generated answers [8]. This LLM guided query reformulation has proven especially helpful for resolving ambiguous or under-specified questions, leading to more robust RAG performance.

Despite these advancements, applying RAG to historical texts introduces unique challenges, including domain-specific language, sparse metadata, and multi linguality. Although prior research has explored automated analysis of historical documents, comprehensive RAG based pipelines specifically tailored for historical speech question answering remain under explored. This project addresses this gap by developing a modular, containerized RAG system that integrates state-of-the-art retrieval and generation techniques for the analysis of historical speeches.

3 Methodology

3.1 System Overview

The system architecture is structured around four core services, each managed with Docker for portability and consistency. The **Elasticsearch** service, based on the official `elasticsearch:8.17.1` image, operates as a single-node instance with security features disabled for simplicity and machine learning capabilities enabled. It exposes port 9200 for external access and uses a dedicated Docker volume (`esdata`) to persist index data, ensuring that stored documents and vector embeddings are retained across restarts.

The **backend** service is built from the local `./backend` directory and runs a FastAPI application. It acts as the API gateway, managing requests from the frontend, retrieving relevant context from Elasticsearch, and generating responses by interacting with the Ollama language model API. The backend also mounts the embedding model and, optionally, the dataset directory for efficient access within the container. Service health is maintained by making the backend dependent on the readiness of both Elasticsearch and Ollama.

The **frontend** is constructed with Next.js (React), built from the `./frontend` directory. It provides a modern user interface for submitting queries and visualizing answers, communicating directly with the backend, and is exposed on port 3000 for user access.

Finally, the **Ollama** service, defined with a custom Dockerfile, hosts large language models such as llama3.2 and Mistral. It exposes its API on port 11434 (mapped to 11500 on the host) and stores all downloaded and managed models in a persistent Docker volume (`ollama_models`). The use of Docker volumes for both Elasticsearch and Ollama ensures reliable and persistent data storage across all services.

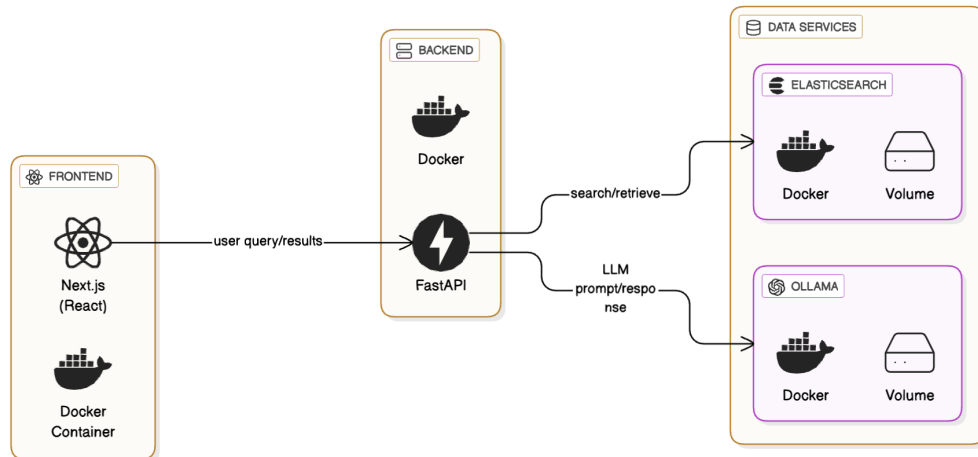


Figure 1: System Architecture

3.2 Retrieval Augmented Generation Pipeline

The system is designed to efficiently answer user questions about historical speeches by integrating advanced document embedding, semantic retrieval with a fallback to hybrid retrieval (if required) or a simple vanilla RAG implementation, with a large language model (LLM) generation. The system allows the user to select the RAG pipeline from the front end.

3.2.1 Basic RAG

The user can select the model LLama3.2 to invoke the basic RAG pipeline that does not use any grader like Document and Hallucination. The pipeline invokes simple RAG call in which document is fetched from the vector database using semantic matches and fed to LLM for final answer generation. The generated answer may be hallucinated as no strict relevancy for document fetched and answer generated is implemented. This interface allow simple user query processing and faster results since few LLM calls are invoked.

3.2.2 Agentic RAG Pipeline with LangGraph

This system utilizes LangGraph, a framework for employing LLM calls and tools in a directed graph structure [1]. Unlike sequential chains, a LangGraph workflow consists of nodes (each performing a specific function like retrieval or grading) and conditional edges that allow branching based on intermediate results [1]. This design allows the system to dynamically adjust retrieval depth, reformulate queries, evaluate generated answers, and recover from failure states adding more flexibility and control.

The pipeline 2 begins with a Route node that classifies the user query into one of four types: simple factual, comparison, complex multi hop, or summarization. For comparison and complex queries, a Pronoun replace and Decomposer node reformulates the input into logically discrete sub queries. Each subquery thereafter along with simple and summarization queries passes through simple or summarize node, respectively that fetches the retrieved documents from the vector database.

The retrieved result is followed by DocumentGrader, which uses an LLM to assess whether the retrieved context sufficiently addresses the query. If the fetched document is not related to the user query, the pipeline triggers Rewrite, where the query is reformulated, and retrieval is attempted again. This self-reflective retrieval loop follows the Self RAG and agentic RAG paradigms [13] [1],

and continues up to a maximum retry threshold. If retries are exhausted, the pipeline transitions to a FailState, which triggers execution of next available sub query.

Once adequate supporting chunks are selected, the GenerateAnswer node produces an answer grounded in the retrieved documents. This answer is then evaluated through two validation nodes: HallucinationGrader, which verifies whether claims are supported by retrieved evidence, and AnswerGrader, which checks relevance and completeness. If deficiencies are detected, the answer generation step is repeated with refined retrieval results. Only validated answers transition to output.

For complex and comparison queries, the system routes sub query responses through either Compare (for contrastive explanation) or ComplexMerge (for multi document synthesis), after which execution terminates at End.

This adaptive RAG pipeline enables the system to self monitor, self correct, and iteratively refine retrieval and response quality, significantly reducing hallucination rates and improving factual grounding for historical speech analysis tasks. Through these interconnected capabilities – query routing, document grading, query rewriting, question decomposition, and answer self checking – the RAG system becomes agentic, i.e., active in diagnosing and correcting its mistakes rather than a passive one-shot pipeline. This yields more reliable answers for users querying historical speeches, especially for complex information needs that require careful reasoning and factual grounding.

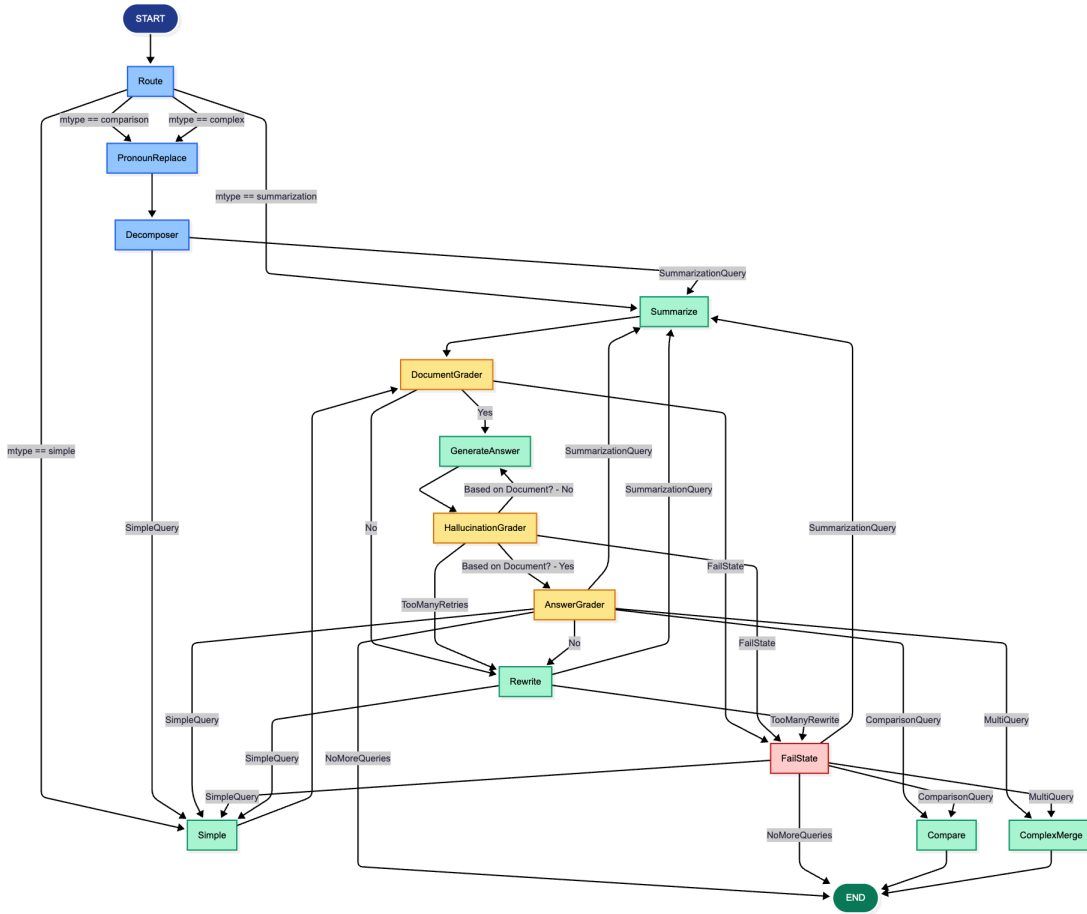


Figure 2: RAG Pipeline with LangGraph

The full pipeline consists of two main stages: *document embedding and indexing* and *inference*

and answer generation.

3.2.3 Document Embedding and Indexing

1. **Document Ingestion and Metadata Extraction:** All speech transcripts are collected in a designated data directory. Each file name encodes metadata, such as the year, month, day, document IDs, context description, and language. Metadata is extracted via a comprehensive regular expression and mapped to fields such as `date`, `document_id`, `additional_id`, and `language`.
2. **Chunking:** Effective document segmentation is essential in Retrieval-Augmented Generation (RAG), as it determines how well relevant text can be retrieved. In our system, we primarily used a recursive character-based text splitter to divide historical speech transcripts into chunks of approximately 500 characters with a 50 character overlap. This approach follows recommended practices for preserving continuity across chunk boundaries and ensures that each chunk remains small enough to fit efficiently into the LLM context window [13]. The typical chunk length (50 words) generally corresponds to a coherent idea or paragraph.

We also experimented with semantic chunking, which attempts to split documents at topic or discourse boundaries (e.g., paragraph or theme transitions) using embeddings or topic shift detection techniques [11]. The expectation was that semantically coherent chunks would improve contextual grounding. However, semantic chunking produced “multi answer chunks” segments that included multiple unrelated informational units (e.g., economic policy + health-care in the same chunk). When such chunks were retrieved for single topic queries, the added extra content reduced retrieval precision and sometimes caused the LLM to include irrelevant details in generated answers.

Furthermore, semantic chunking led to highly variable chunk sizes, where longer chunks risked exceeding the LLM’s context limit and required truncation. In contrast, fixed size recursive chunking provided stable, predictable chunk boundaries and reduced the chance of losing important details due to overly broad grouping.

This outcome aligns with findings that smaller, focused chunks improve retrieval relevance because each chunk is more likely to contain a single discrete fact or argument, which maps more directly to user queries. The comparative performance of recursive, semantic, and hybrid chunking strategies is presented in Section 4 (Evaluation). Figure 3 shows the chunking strategies in practise.

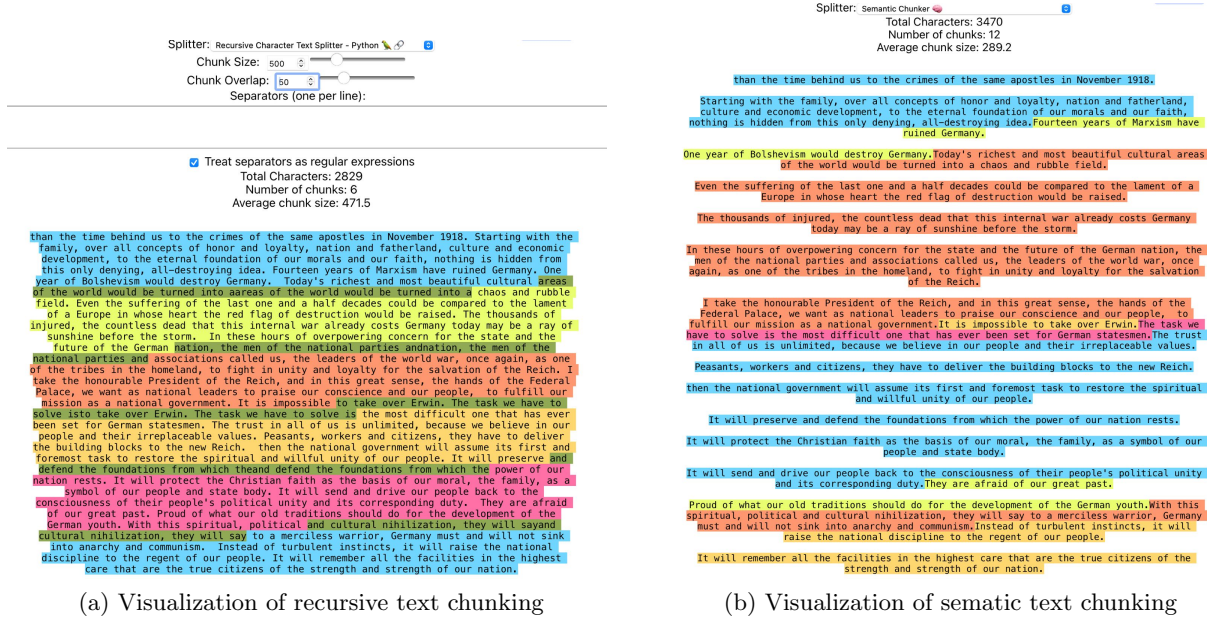


Figure 3: (a) Recursive Text Character Splitter. (b) Semantic Chunking.

3. Embedding Generation: To support cross-lingual retrieval across English and German speech transcripts, the system uses Arctic-Embed 2.0, a multilingual embedding model released by Snowflake in late 2024 [18]. The model generates 1024 dimensional dense embeddings and supports 74 languages, allowing a query in one language (e.g., English) to retrieve semantically aligned passages in another (e.g., German) [2]. Unlike earlier multilingual embedding models, which often sacrificed English semantic quality for multilingual coverage, Arctic-Embed 2.0 maintains strong English performance while improving retrieval accuracy in German and other languages [2].

All speech chunks and queries are encoded locally using Arctic-Embed via Ollama, ensuring data privacy and no external API dependence. The resulting vectors are stored in Elastic-search’s dense vector index, enabling cosine similarity based retrieval. Metadata fields (e.g., context, language, date) support filtering and future hybrid retrieval. The retrieved chunks return both text and similarity scores, which are later used in the pipeline’s LLM based grading stages.

Excerpt from Elastic search custom search

```
return {
  "script_score": {
    "query": {
      "bool": {
        "must": [{"term": {"join_field": "chunk"}}],
        "filter": []
      }
    },
    "script": {
      "source": """
        double score = cosineSimilarity(params.query_vector, 'embedding');
        """
    }
  }
}
```

```

    }
  }
}

```

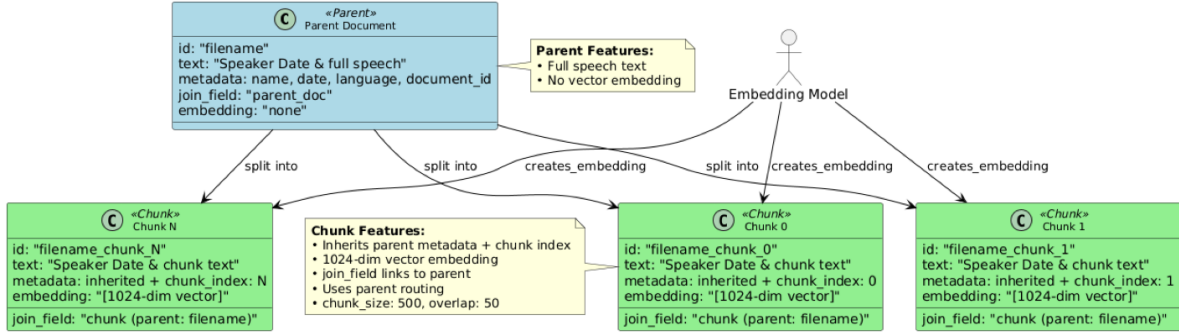


Figure 4: Parent Child Retriever

4. **Indexing in Elasticsearch:** The system makes use of Parent Document Retriever (PDR) during indexing that balance the need for detailed information with the necessity of preserving broader context in document retrieval systems. It achieves this by structuring documents into smaller segments, or “child chunks,” while maintaining references to their original, larger “parent” documents. During retrieval, the system first identifies relevant child chunks based on a user’s query and then retrieves the corresponding parent documents to provide comprehensive context [4]. Both the full document and its individual chunks are indexed in Elasticsearch. The parent document stores the full text and metadata (without embeddings), while each chunk is indexed as a child document with its embedding, metadata (including its chunk index), and a join field linking it to the parent. The Elasticsearch index mapping includes dense vector fields and supports cosine similarity for fast vector search. Figure 4 illustrates this indexing strategy.

3.2.4 Inference and Answer Generation

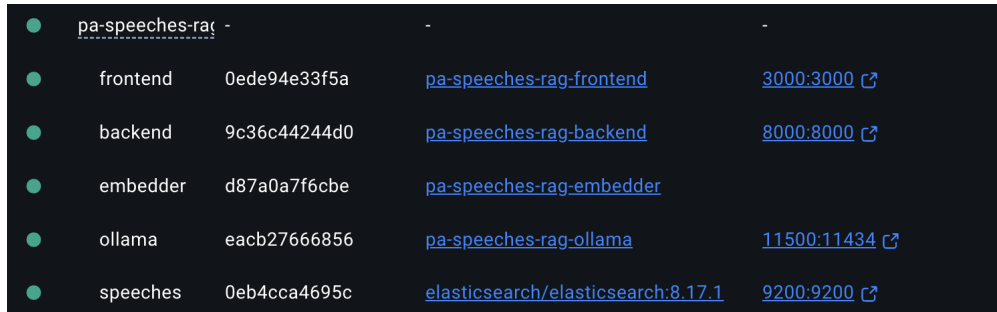
1. **User Query Encoding:** When a user submits a question through the frontend, the question text is embedded using the same SentenceTransformer model as for indexing.
2. **Document Retrieval:** The system performs a cosine similarity search over the vector database based on the embedded user query:
 - A script score combines cosine similarity of the query embedding and the chunk embeddings, L2 distance, and traditional keyword match scoring in a hybrid search.
 - For semantic search just the cosine similarity is used.
 - Additional filters can be applied based on language or date if specified in the query.
3. **Prompt Construction:** Retrieved chunks are concatenated with the user question to form a comprehensive prompt. The prompt includes strict instructions, asking the LLM to answer only using the provided context and to refrain from guessing if the context is insufficient. There are multiple different prompts for document grader, hallucination grader, route node, complex and comparison queries.
4. **LLM Answer Generation:** The prompt is submitted to a large language model (e.g., Llama3.2 or qwen2.5:7b) hosted in an Ollama container. The model generates a context aware answer grounded in the retrieved document chunks.

5. **Post-processing and Reporting:** The system processes the answer, formats it for clarity, and produces a detailed Markdown report. The report includes:
 - The user's original question,
 - The retrieved chunks with query terms highlighted,
 - Source metadata (document IDs, titles, dates),
 - The LLM's generated answer.

This information is returned to the frontend for user display. Additionally, the backend exposes endpoints for retrieving available LLM models and other metadata.

3.3 Backend Architecture

The backend is implemented in Python using the FastAPI framework, providing a RESTful API that orchestrates the core retrieval-augmented generation (RAG) pipeline. It handles requests from the frontend, encodes user queries into embeddings using the Snowflake Arctic embed model, retrieves relevant document chunks from Elasticsearch using a hybrid search strategy, and communicates with large language models hosted via the Ollama container for answer generation. The backend also includes endpoints for model selection, result formatting, and metadata filtering. All major pipeline components including embedding generation, vector search, prompt construction, and answer post-processing are modularized for flexibility and reproducibility.



●	pa-speeches-rag	-	-	-
●	frontend	0ede94e33f5a	pa-speeches-rag-frontend	3000:3000 ↗
●	backend	9c36c44244d0	pa-speeches-rag-backend	8000:8000 ↗
●	embedder	d87a0a7f6cbe	pa-speeches-rag-embedder	
●	ollama	each27666856	pa-speeches-rag-ollama	11500:11434 ↗
●	speeches	0eb4cca4695c	elasticsearch/elasticsearch:8.17.1	9200:9200 ↗

Figure 5: Overview of running backend, frontend, database, and LLM containers in the RAG pipeline.

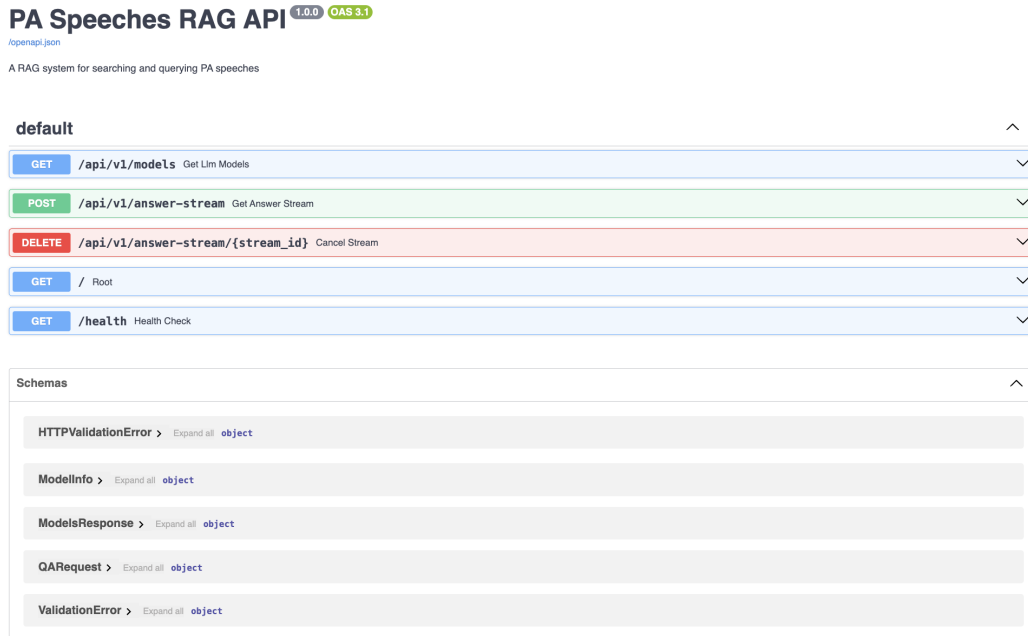


Figure 6: Backend API endpoints provided by the FastAPI service for the RAG system.

3.4 Frontend Architecture

The frontend is built with Next.js (React), offering a user-friendly web interface for question answering over historical speeches. Users can submit queries, select the desired language model, and view retrieved answers alongside supporting context. The interface communicates with the backend via RESTful API calls, dynamically displays answer confidence and metadata, and highlights relevant terms within retrieved chunks. The design emphasizes clarity and transparency, providing users with both the generated answer and the evidence used for its generation.

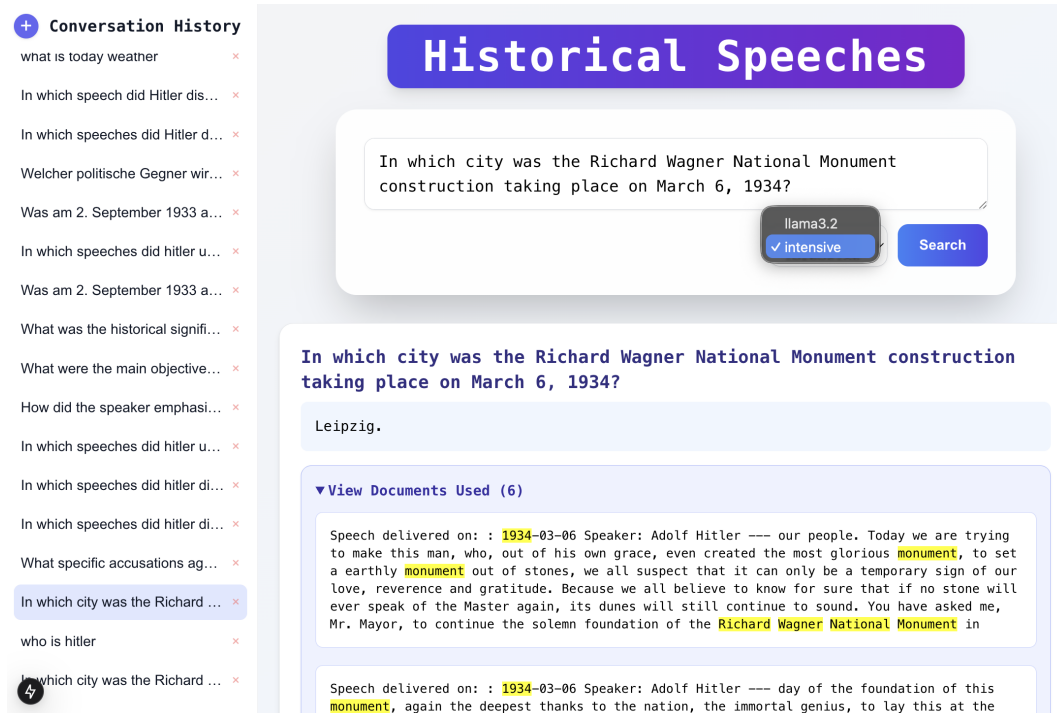


Figure 7: User Interface

3.5 Data Preparation

The dataset used in this project was provided by our mentor and consists of historical speech transcripts in text format. Each file includes metadata such as date, language, and document identifiers, which were parsed either from the file names or embedded file headers. Prior to embedding, all documents were preprocessed to remove formatting inconsistencies and ensure clean text input. The preprocessed data was then systematically chunked and embedded as described in the methodology section, enabling efficient retrieval and robust downstream analysis.

3.6 Model Selection and Configuration

According to Vladika and Matthes (2025) for a QA based Retrieval Augmented System for biomedical field, the models like Qwen has the optimum result with the context size lesser than 15 [17]. Our experiment too show the same result as in section 4 - Evaluation. Therefore, for language generation, the system supports multiple large language models served via Ollama, including **llama3.2** for Basic RAG pipeline and **Qweb2.5:7b** for agentic RAG pipeline. The choice of LLM can be selected at query time via the frontend interface. All models are containerized and managed using Docker to ensure consistency and reproducibility across different environments.

3.7 Experimental Setup

All experiments were conducted on a remote machine equipped with an NVIDIA A100 (80 GB) GPU. The software environment was containerized and managed using Docker, while service orchestration and reproducibility were ensured through Docker Compose.

The **Elasticsearch** service (`elasticsearch:8.17.1`) was configured as a single-node instance with 3 GB of memory allocated, and machine learning features enabled. Data persistence was

achieved via a dedicated Docker volume (`esdata`). The **backend** service, built from a local Dockerfile, managed communication with Elasticsearch and the language model, and was exposed on port 8000. The backend was provided access to the embedding model and data directories via Docker volumes.

The **frontend** was implemented in Next.js (React), built and served from its own container, and communicated with the backend API. It was exposed on port 3000 and shared its source code directory via a volume for live development and testing.

For LLM inference, the **Ollama** service was run in a container using a custom Dockerfile, with all language models stored in a persistent Docker volume (`ollama_models`). The service was exposed on port 11434 (mapped to 11500 on the host for convenience).

All containers were started and managed via Docker Compose, with explicit `depends_on` conditions to enforce proper service initialization order. Health checks were enabled for Elasticsearch to ensure it was available before other services started.

Excerpt from `docker-compose.yml`

```
services:
  elasticsearch:
    image: docker.elastic.co/elasticsearch/elasticsearch:8.17.1
    container_name: speeches
    environment:
      - discovery.type=single-node
      - xpack.security.enabled=false
      - xpack.ml.enabled=true
      - network.host=0.0.0.0
      - ES_JAVA_OPTS=-Xms2g -Xmx2g
    ports:
      - "9200:9200"
    mem_limit: 3g
    volumes:
      - /data/PA-Speeches-RAG/esdata:/usr/share/elasticsearch/data
      - /data/PA-Speeches-RAG/tmp:/tmp
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:9200/_cluster/health"]
      interval: 30s
      timeout: 10s
      retries: 5
  backend:
    build: ./backend
    container_name: backend
    command: uvicorn main:app --host 0.0.0.0 --port 8000
    #runtime: nvidia
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              device_ids: ["1"]
              capabilities: [gpu]
    environment:
      - NVIDIA_VISIBLE_DEVICES=1
      - ES_HOST=http://elasticsearch:9200
```

```

- INDEX_NAME=re_updated_index
- EMBEDDING_MODEL=/app/embedding_model
- OLLAMA_BASE_URL=http://ollama:11434
- LLM_MODELS=llama3.2:llama3.2,intensive:intensive
- NVIDIA_VISIBLE_DEVICES=GPU-8cc0c2da-a5b0-02c5-eae2-b2b54b810624
- PYTHONUNBUFFERED=1
- LOG_LEVEL=debug
volumes:
- ./backend:/app
- /data/PA-Speeches-RAG/embedding_model:/app/embedding_model
- /data/PA-Speeches-RAG/data:/app/data:ro
- /data/PA-Speeches-RAG/semantic_model:/app/semantic_model
ports:
- "8000:8000"
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
  interval: 10s
  timeout: 5s
  retries: 5
depends_on:
  elasticsearch:
    condition: service_healthy
  ollama:
    condition: service_healthy
frontend:
  build: ./frontend
  container_name: frontend
  environment:
    - NEXT_PUBLIC_BACKEND_URL=http://backend:8000
    - NEXT_PUBLIC_API_TIMEOUT=600000
  volumes:
    - ./frontend:/app
  ports:
    - "3000:3000"
  depends_on:
    backend:
      condition: service_healthy
ollama:
  build:
    context: .
    dockerfile: Dockerfile.ollama
  container_name: ollama
  platform: linux/amd64
  #runtime: nvidia
  deploy:
    resources:
      reservations:
        devices:
          - driver: nvidia
            device_ids: ["1"]
            capabilities: [gpu]
  environment:
    - NVIDIA_VISIBLE_DEVICES=1
  ports:

```

```

- "11500:11434"
volumes:
- /data/PA-Speeches-RAG/ollama_models:/root/.ollama
restart: unless-stopped
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost:11434"]
  interval: 5s
  timeout: 5s
  retries: 20
embedder:
  build: ./embedder
  container_name: embedder
  runtime: nvidia
  environment:
    - ES_HOST=http://elasticsearch:9200
    - INDEX_NAME=re_updated_index
    - EMBEDDING_MODEL=/app/embedding_model
    - NVIDIA_VISIBLE_DEVICES=1
  volumes:
    - ./backend:/app
    - /data/PA-Speeches-RAG/embedding_model:/app/embedding_model
    - /data/PA-Speeches-RAG/data:/app/data:ro
    - /data/PA-Speeches-RAG/semantic_model:/app/semantic_model
  depends_on:
    elasticsearch:
      condition: service_healthy

```

4 Evaluation

4.1 Evaluation Setup

The evaluation aims to examine how different chunking strategies influence retrieval relevance, grounding accuracy, semantic coherence, and overall system efficiency within the Retrieval-Augmented Generation (RAG) pipeline [6]. Specifically, three chunking strategies are compared: **Semantic Chunking**, **Recursive Character Splitting**, and **Hybrid Chunking**. These strategies differ in how they partition the source documents prior to embedding, thereby affecting the granularity and contextual completeness of retrieved text.

The evaluation was conducted on a evaluation dataset curated from historical speech transcripts. Each speech was preprocessed, segmented using the respective chunking strategy, embedded using Arctic-Embed 2.0 [18], and indexed in Elasticsearch [15]. The RAG pipeline itself was implemented and orchestrated using LangChain [16], ensuring modular and reproducible workflow execution.

In addition, the retrieval process is augmented by two internal evaluators: the **Document Grader (DG)** and the **Hallucination Grader (HG)**. The DG evaluates whether the retrieved textual evidence is relevant to the user query, while the HG verifies that the generated answer remains grounded in the retrieved evidence and does not introduce unsupported statements. A DG-fail indicates retrieval misalignment, whereas an HG-fail indicates hallucination or semantic drift during answer generation.

In as separate execution, the experiment also incorporated a **Query Expansion with Hypothetical Answer Module** to test if the LLM generated answer for the user question helps in effective document retrieval. The Query Expansion Module transform the user question to possi-

ble excerpt of speech for better retrieval. The experiment with Query Expansion used Recursive Splitter Chunking strategy.

All experiments were performed in a fully containerized environment to ensure consistency and reproducibility across runs.

4.2 Evaluation Set

To systematically assess performance, a benchmark evaluation set consisting of 100 question–answer pairs was constructed from the historical speech corpus. Each question was paired with a gold-standard reference answer generated using the same Llama 3.2 model under grounded conditions, ensuring stylistic and semantic comparability. During evaluation, the RAG system used the identical model to generate answers, enabling a controlled, model-consistent comparison.

For each query, the generated answer and the reference answer were tokenized and evaluated using standard natural language generation metrics. **BLEU** was computed using the sentence-level implementation with smoothing to handle short responses [12]. **ROUGE-L** was calculated using the `rouge_score` library with stemming enabled to capture sequence-based recall [7]. **BERTScore** was computed using a multilingual, context-aware model to measure semantic similarity between the generated and reference responses [19]. Additionally, the **average response time** was recorded for each evaluation instance to assess pipeline efficiency.

All evaluation procedures, including retrieval execution, grading, metric computation, and logging, were automated using Python to guarantee reproducibility and consistent comparison across chunking strategies and model configurations.

4.3 Metrics

We use the following evaluation metrics:

- **BLEU** [12] — n -gram precision similarity with the reference.
- **ROUGE-L** [7] — longest-common-subsequence recall.
- **BERTScore** [19] — semantic similarity via contextual embeddings.
- **Average Response Time** — total retrieval + generation latency.
- **DG Pass/Fail** — retrieval relevance.
- **HG Pass/Fail** — hallucination or grounding correctness.

4.4 Quantitative Results

Strategy	BLEU	ROUGE-L	BERTScore	Avg. Time (s)
Semantic Chunking	0.0247	0.0841	0.7226	7.21
Hybrid Chunking	0.0524	0.1834	0.7687	4.80
Recursive Splitter	0.0501	0.1748	0.7667	4.78
Query Expansion	0.0558	0.1843	0.7673	18.18

Table 1: Comparison of performance across chunking strategies.

Strategy	DG Fail	HG Fail	Total Queries	Accuracy (%)
Semantic	43	1	100	56
Hybrid	19	4	100	77
Recursive	16	2	100	82
Query Expansion	20	1	100	79

Table 2: DG/HG failure statistics across chunking strategies.

4.5 Visual Summaries

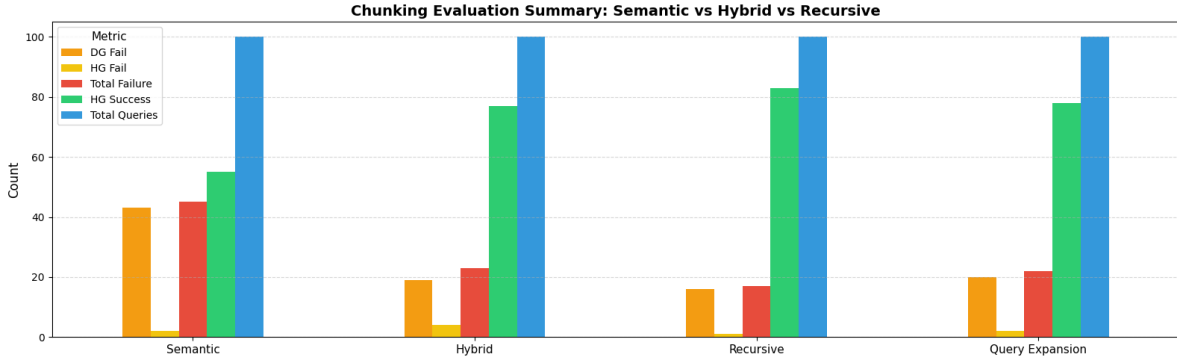


Figure 8: DG/HG outcome comparison for Semantic, Hybrid, and Recursive chunking.

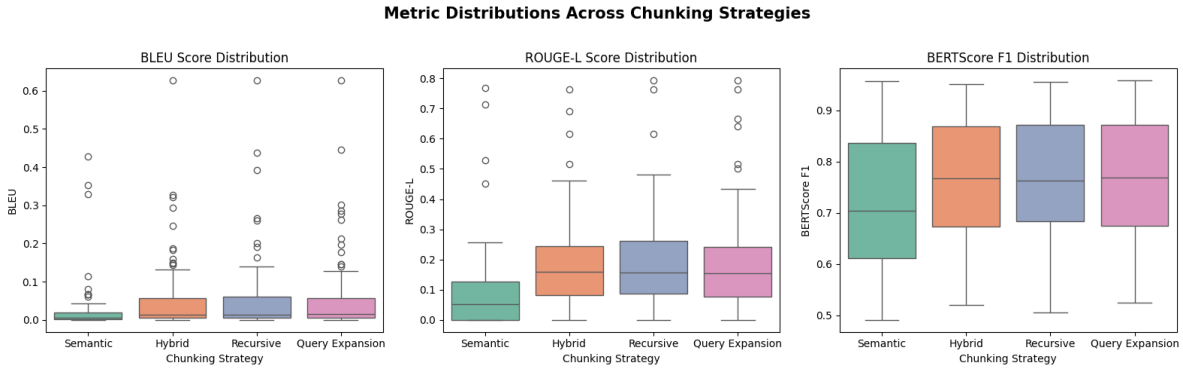


Figure 9: Distribution of BLEU, ROUGE-L, and BERTScore across chunking strategies.

TimeAcross Chunking Strategies

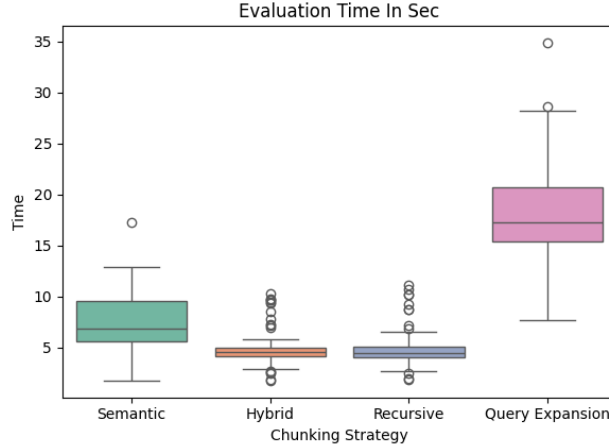


Figure 10: Evaluation time across chunking strategies.

4.6 Discussion

The evaluation results clearly show that the choice of chunking strategy has a direct and measurable impact on retrieval relevance, grounding behavior, and final answer quality in the RAG pipeline. Although all three strategies rely on the same embedding model and retrieval mechanism, their differences in text segmentation significantly influence how well the retrieved context aligns with the user query.

Semantic Chunking preserves the natural flow of ideas by splitting text at paragraph or discourse boundaries. However, historical speeches often consist of long ideological monologues, resulting in extremely large chunks (up to 22,904 characters in our dataset). Such oversized chunks contain multiple topics, making it difficult for the retriever to isolate the correct context segment, which is reflected in the **highest DG-fail rate (43%)**. The low BLEU and ROUGE-L scores further indicate that answers generated from these large, multi-theme chunks tend to be generic or only loosely aligned to the specific question. Additionally, semantic chunking incurs the highest latency due to increased embedding and vector comparison costs.

In contrast, the **Recursive Character Splitter** produces consistently sized chunks (average ~ 389 characters), which improves the precision of retrieval by narrowing the contextual scope of each chunk. This strategy ensures that retrieved text is both relevant and concise, resulting in the **lowest DG-fail rate (16%)** and strong improvements in BLEU, ROUGE-L, and BERTScore metrics. Because chunk boundaries are determined by character length rather than semantic cues, some chunks may begin or end mid-sentence. However, this did not significantly impair grounding: the **HG-fail rate remained low (2%)**, indicating that the LLM could effectively infer local continuity without hallucinating. The Recursive approach also delivered the **lowest average response time (4.78 s)**, making it both effective and efficient.

The **Hybrid Chunking** strategy attempted to combine both advantages by first applying semantic segmentation and then refining larger segments with the Recursive splitter. While this reduced chunk size variability and improved coherence compared to pure semantic chunking, its performance did not match the precision of the fully Recursive approach. The DG-fail rate (19%) and HG-fail rate (4%) suggest that although the retrieved evidence was often relevant, it sometimes contained excess context that introduced mild semantic drift. Performance metrics improved over semantic chunking but were slightly below those of the Recursive approach. Latency performance

was similar to the Recursive approach, confirming that chunk size regularity contributes to faster retrieval and embedding.

In summary, the Recursive Character Splitter demonstrates the best overall performance. It achieves the optimal balance between retrieval accuracy, semantic focus, grounding reliability, and computational efficiency. The results confirm that *finer-grained, consistently sized chunks enable more precise and context-aware retrieval*, which directly reduces hallucination risk and improves answer quality in RAG systems.

1. **Recursive Splitter yields the most accurate retrieval**, producing the lowest DG-fail rate.
2. **Grounding remains stable with minimal hallucination**, as shown by the lowest HG-fail rate.
3. **Answer quality is highest when chunk granularity is controlled**, supporting compact, focused reasoning.
4. **Latency is minimized when chunk size is consistent**, improving real-time usability.

Therefore, the Recursive Character Splitting strategy is identified as the **most effective and practical** chunking approach for RAG-based analysis of long, rhetorically dense historical speeches.

And hence, the **Query Expansion with Hypothetical Answer** was performed using the Recursive Text Splitter Chunking strategy, but the result after the experiments were not satisfactory. The execution time increased tremendously and the system failure (DG, HG) was on the higher side too. Moreover, the system response with summarization and comparison queries were not accurate and misses the information significantly. Also, LLM introduces some vague excerpt in response which never existed. Therefore, the Query Expansion with Hypothetical Answer does not seem promising.

5 Conclusion

This project demonstrates the effectiveness of a retrieval-augmented generation (RAG) system for answering questions about historical speeches. By combining dense retrieval using semantic embeddings with large language models, our approach produces context-aware and semantically relevant answers, even when the wording differs from reference responses. The system is flexible, supports multiple languages, and is easily deployable using Docker.

Our main contributions include the design and implementation of a complete RAG pipeline, the creation of a custom evaluation set, and a thorough quantitative and qualitative analysis of system performance. While the results are promising, challenges remain in improving exact answer matching, handling more complex queries, and further reducing response time.

6 Code and Resources

All code, scripts, and documentation are available at: <https://github.com/umr-ds/PA-Speeches-RAG.git>

References

- [1] Langgraph rag: How to build agentic retrieval-augmented generation. <https://www.leanware.co/insights/langgraph-rag-agentic>, 2025. Accessed: 26 June 2025.
- [2] Snowflake/arctic embed model - hugging face. <https://huggingface.co/Snowflake/snowflake-arctic-embed-l-v2.0>, 2025. Accessed: 2 February 2025.
- [3] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-rag: Learning to retrieve, generate, and critique through self-reflection, 2023.
- [4] DhanushKumar. Parent document retriever for rag. <https://medium.com/@danushidk507/rag-ix-parent-document-retriever-a49450a482ab>, 2025. Accessed: 2 April 2025.
- [5] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C. Park. Adaptive-rag: Learning to adapt retrieval-augmented large language models through question complexity, 2024.
- [6] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks, 2021.
- [7] Chin-Yew Lin. Rouge: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, 2004.
- [8] Xinbei Ma, Yeyun Gong, Pengcheng He, Hai Zhao, and Nan Duan. Query rewriting for retrieval-augmented large language models, 2023.
- [9] Suchitra Malimbada. Chunking in agentic rag. <https://pub.towardsai.net/all-you-need-to-know-about-chunking-in-agentic-rag-bdc34c21a6e6>, 2025. Accessed: 11 October 2025.
- [10] Medium. Vanilla rag vs agentic rag. <https://bytebridge.medium.com/vanilla-rag-vs-agentic-rag-4d756ddb611f>, 2025. Accessed: 01 September 2025.
- [11] Plaban Nayak. Semantic chunking for rag. <https://medium.com/the-ai-forum/semantic-chunking-for-rag-f4733025d5f5>, 2025. Accessed: 26 August 2025.
- [12] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. Bleu: A method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 311–318, 2002.
- [13] Sejal Sharma. Comprehensive agentic rag. <https://sajalsharma.com/posts/comprehensive-agentic-rag/>, 2025. Accessed: 15 October 2025.
- [14] Ishneet Sukhvinder Singh, Ritvik Aggarwal, Ibrahim Allahverdiyev, Muhammad Taha, Aslihan Akalin, Kevin Zhu, and Sean O’Brien. Chunkrag: Novel llm-chunk filtering method for rag systems, 2025.
- [15] Elasticsearch Team. Elasticsearch documentation. <https://www.elastic.co/docs>, 2025. Accessed: 01 January 2025.
- [16] LangChain Team. Langchain documentation. <https://www.langchain.com/>, 2025. Accessed: 09 August 2025.

- [17] Juraj Vladika and Florian Matthes. On the influence of context size and model choice in retrieval-augmented generation systems, 2025.
- [18] Puxuan Yu, Luke Merrick, Gaurav Nuti, and Daniel Campos. Arctic-embed 2.0: Multilingual retrieval without compromise, 2024.
- [19] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. BERTScore: Evaluating text generation with BERT. In *International Conference on Learning Representations (ICLR)*, 2020.

A Appendix

A.1 Hardware and Software Used

All experiments were conducted on a local Apple Mac computer (Apple Silicon M4), with Docker configured to use 16 GB of RAM. The host operating system was macOS. The evaluation set was conducted on a remote machine equipped with an NVIDIA A100 (80 GB) GPU. The main components and dependencies are:

- **Docker** and **Docker Compose** for service orchestration
- **Python** (version 3.10) for backend, data processing, and evaluation scripts
- **FastAPI** for RESTful backend API
- **Elasticsearch** (version 8.17.1) for document indexing and retrieval
- **Node.js** and **Next.js** for the frontend
- **Ollama** (custom Docker image) for running large language models
- **SentenceTransformers** library for embedding generation
- **Numpy**, **Pandas**, **Matplotlib**, and **Seaborn** for evaluation and visualization

All major components were containerized to ensure reproducibility and easy deployment. Please refer to the `docker-compose.yml` for detailed service configuration.

A.2 Folder Structure

The overall folder organization of the project is as follows:

```
project-root/  
  backend/  
  frontend/  
  embedder/  
  ollama_models/  
  semantic_model/  
  data/  
  embedding_model/  
  docker-compose.yml  
  README.md
```

A.3 Deployment Instructions

To build and run the complete system, follow these steps:

1. **Clone the repository:**

```
git clone <your-repo-url>  
cd <project-root>
```

2. Prepare the data and models:

- (a) Place the historical speech data (text files) in the `data/` directory.
- (b) Download or copy the embedding model files into the `embeddingmodel/` directory.

3. Install Docker and Docker Compose:

```
# On macOS:  
brew install docker docker-compose
```

Or use official installers from <https://docs.docker.com/get-docker/>

4. Build and start all services:

```
docker compose up --build
```

This command builds the backend and frontend images and starts all containers (Elasticsearch, backend, frontend, and Ollama) as defined in `docker-compose.yml`.

5. Access the system:

- The frontend web interface is available at `http://localhost:3000`
- The backend API runs at `http://localhost:8000`

6. (Optional) Stop all services:

```
docker compose down
```

For additional configuration, troubleshooting, or advanced usage, please refer to the `README.md` file in the project root.

A.4 API Endpoints

The backend exposes several REST API endpoints for interaction with the RAG system:

- **POST /answer**
Accepts a JSON payload with a question and model name. Returns the generated answer, relevant context chunks, and evaluation metadata.

Request Example:

```
{  
  "question": "What is the main topic of the speech?",  
  "model": "llama3.2"  
}
```

Response Example:

```
{
  "answer": [...],
  "llm_answer": "...",
  "markdown": "...",
}
```

- GET /models

Returns a list of available LLM models for selection in the frontend. **Response Example:**

```
{
  "models": [
    {"label": "Llama 3.2", "value": "llama3.2"},
    {"label": "intensive", "value": "qwen2.5:7b"}
  ]
}
```

A.5 Sample Data or Outputs

Below is an example of a question-answer evaluation record from the system:

```
{
  "filename": "1955_EN_Churchill.txt",
  "date": "1955-04-05",
  "language": "english",
  "question": "What was Churchill's main message in this speech?",
  "gold_answer": "text",
  "model": "llama3.2",
  "bleu": 0.42,
  "rougeL": 0.56,
  "bertscore_f1": 0.88,
  "eval_time_sec": 12.7
}
```